Week 3 Submission

Peer-to-Peer Lending Matchmaking System

Regd No	25091A32G9, 25091A32K4
Problem Statement No	105
Title	Peer-to-Peer Lending Matchmaking System
Technology	Python, Flask, SQLite, NetworkX, Pandas/NumPy, ReportLab, OpenPyXL, Chart.js, Bootstrap, unittest/pytest
Data	Academic simulation using simulated test data - no real-money transactions

Where to run the code (read this first)

- **Where:** on your own laptop. Install **Python 3.10 or newer** from python.org (tick *Add Python to PATH*) and **VS Code**. Everything below is typed in the **VS Code terminal** (menu Terminal > New Terminal, or Ctrl+`).
- **Which folder:** open your **p2p_lending** folder (the one that contains **app.py**) with File > Open Folder. Every command must be run from this folder, not from a sub-folder, not in IDLE.
- **One-time setup:** run `python -m venv venv`, then activate it - Windows: `venv\Scripts\activate` (if PowerShell blocks it use Command Prompt or run `Set-ExecutionPolicy -Scope Process Bypass`), macOS/Linux: `source venv/bin/activate`. Then `pip install -r requirements.txt`.
- **Adding the files of a week:** in VS Code right-click the folder > New File, type the exact path shown in the heading (for example `routes/profiles.py`), paste the code. **[NEW]** = create the file, **[REPLACE]** = overwrite the old file completely, **[APPEND]** = paste at the very end of the existing file.
- **Web pages:** run `python app.py`, then open `http://127.0.0.1:5000` in your browser (stop the server with Ctrl+C). The pages load Bootstrap/Chart.js from the internet, so stay online.
- **Keep the weeks cumulative:** Week N needs the files of Weeks 1 to N-1. Your Week 1 zip is the starting point and none of its files is deleted later (only app.py and templates/index.html are replaced in Week 2, and config.py gets lines appended in Week 3).
- **Demo scripts** (`demo_weekN.py`) create their own throw-away databases in the data folder, so they never disturb your real database. To see the same demo data in the web pages: stop the server, delete `data/p2p_lending.db`, run `python demo_data.py`, start the server again.

Week 3: Risk and Compatibility Scoring

Calculate a transparent rule-based borrower-risk score, classify borrowers, calculate matching scores, reject pairs that violate hard constraints, and store score components and explanations. **Working output: compatibility-scoring engine.**

Assignment steps and where they are done

Assignment step	Implemented in
1. Rule-based borrower-risk score using simulated data	scoring.py: RiskScorer.score() - 4 weighted components, 0-100
2. Classify borrowers low / medium / high	RiskScorer.classify() with the thresholds 70 and 45 from config.py
3. Individual matching scores	CompatibilityScorer.fits() - Risk fit, Amount fit, Term fit, Return fit
4. Reject pairs that violate hard constraints	CompatibilityScorer.check_constraints() - 7 rules with error codes
5. Store score components and explanations	score_all() writes risk_scores (JSON components) and matches (components + explanation)
6. Display why a match was recommended or rejected	/scoring/pairs (every pair with its explanation) and /scoring/risk (risk explanation)

Files of this week

File	Action	Purpose
config.py	APPEND	Adds the risk-score weights, partial-funding fraction and (for Week 5) repayment constants
scoring.py	NEW	RiskScorer, CompatibilityScorer, score_all()
queries.py	NEW	list_pairs(), list_risk_scores() for the screens
routes/scoring.py	NEW	Pages /scoring/run, /scoring/risk, /scoring/pairs
demo_week3.py	NEW	Console demo
tests/test_week3.py	NEW	20 tests

Formulas used

Item	Formula
Risk score (0-100, higher = safer)	credit: (score-300)/600 x 40 + debt-to-income: (1 - min(DTI/0.60, 1)) x 30 + history: min(years,10)/10 x 15 + loan-to-income: (1 - min(amount/annual income, 1)) x 15
Risk class	score >= 70 LOW (9%) ; score >= 45 MEDIUM (14%) ; otherwise HIGH (20%)
Risk fit (40 points)	lender appetite equals borrower class: 100% ; one level safer than appetite: 70% ; two levels: 40%
Amount fit (25 points)	average of: how central the amount is in the lender's loan range (100% centre, 50% edge) and 1 - 0.5 x (amount / lender's available funds)
Term fit (15 points)	100% at the middle of the lender's duration range, falling to 60% at the edges
Return fit (20 points)	50% + 50% x position of the offered rate inside the lender's expected return range (higher return = better)
Compatibility score	Risk fit + Amount fit + Term fit + Return fit (maximum 100)
Affordability rule	(existing obligations + instalment of this loan) / monthly income must not exceed 60%

How to run and check this week

1. `python tests/test_week3.py` - 20 tests, all ok
2. `python demo_week3.py` - risk scores, eligible lenders, rejection reasons, explanations
3. `python app.py` - open /scoring/risk and /scoring/pairs, press 'Run scoring' (needs data: see 'Demo scripts' on the first page)

Expected output of demo_week3.py

```
Scoring summary: {'requests_scored': 7, 'lenders': 5, 'pairs': 35, 'eligible': 11, 'rejected': 24, 'seconds': 0.0043}

=== Borrower risk scores ===
Request #1 Ravi Kumar   amount=   50,000  score= 78.29  -> LOW
Request #2 Meena Iyer   amount=   20,000  score= 44.25  -> HIGH
Request #3 Sunil Verma  amount=   80,000  score= 57.28  -> MEDIUM
Request #4 Anita Das    amount=   40,000  score= 36.33  -> HIGH
Request #5 Kiran Rao    amount=   15,000  score= 91.68  -> LOW
Request #6 Kiran Rao    amount=  120,000  score= 90.22  -> LOW
Request #7 Pooja Nair   amount=  280,000  score=  59.1  -> MEDIUM

=== Eligible lender candidates per request (best first) ===
Request #1 Ravi Kumar   <- Conservative Carla  score= 87.68 (risk 40.0 + amount 21.28 + term 11.4 + return 15.0)
Request #1 Ravi Kumar   <- Balanced Bala       score= 69.09 (risk 28.0 + amount 20.38 + term 10.71 + return 10.0)
Request #2 Meena Iyer   <- Aggressive Arjun     score= 86.52 (risk 40.0 + amount 18.89 + term 9.63 + return 18.0)
Request #3 Sunil Verma  <- Balanced Bala       score= 94.03 (risk 40.0 + amount 21.56 + term 14.14 + return 18.33)
Request #3 Sunil Verma  <- Aggressive Arjun     score= 73.35 (risk 28.0 + amount 19.93 + term 13.42 + return 12.0)
Request #3 Sunil Verma  <- Cohort Chitra        score= 67.25 (risk 28.0 + amount 15.25 + term 12.0 + return 12.0)
Request #5 Kiran Rao    <- Conservative Carla  score= 85.28 (risk 40.0 + amount 18.88 + term 11.4 + return 15.0)
Request #5 Kiran Rao    <- Small Sita          score= 72.01 (risk 28.0 + amount 19.87 + term 14.14 + return 10.0)
Request #5 Kiran Rao    <- Balanced Bala       score= 67.73 (risk 28.0 + amount 19.02 + term 10.71 + return 10.0)
Request #6 Kiran Rao    <- Conservative Carla  score= 82.93 (risk 40.0 + amount 18.93 + term 9.0 + return 15.0)
Request #6 Kiran Rao    <- Balanced Bala       score= 71.31 (risk 28.0 + amount 20.88 + term 12.43 + return 10.0)

=== Why were pairs rejected? (request #4 - Anita Das) ===
Conservative Carla: REJECTED: Borrower is high risk but the lender only accepts up to low risk. Rate 20% for high risk is outside the
    lender's expected return 7-11%. Obligations plus the new instalment (3,705) are 72% of income, above the 60% limit.
Balanced Bala: REJECTED: Borrower is high risk but the lender only accepts up to medium risk. Rate 20% for high risk is outside the
    lender's expected return 9-15%. Obligations plus the new instalment (3,705) are 72% of income, above the 60% limit.
=== Example explanation of a recommendation ===
ELIGIBLE: compatibility 94.03/100 = risk fit 40.0/40 (lender appetite medium, borrower medium) + amount fit 21.56/25 (loan 80,000 vs
    lender band and funds) + term fit 14.14/15 (24 months) + return fit 18.33/20 (rate 14% in lender's 9-15% range).

=== Example risk explanation ===
Risk score 78.29/100 -> LOW risk (low >= 70, medium >= 45). Credit score 780: 32.0/40. Debt-to-income 13%: 23.33/30. Credit history 6
    years: 9.0/15. Loan is 7% of annual income: 13.96/15. Offered interest rate for low risk: 9% per year.
```

[APPEND] config.py (add these lines at the END of your existing config.py)

```
# ============================================================================
# ---- Added in Week 3 (risk and compatibility scoring) ----------------------
# ============================================================================
# Borrower risk score (0-100, higher = safer). The four parts add up to 100.
RISK_SCORE_WEIGHTS = {"credit": 40, "dti": 30, "history": 15, "loan_to_income": 15}

# Partial funding (optional enhancement): a lender must hold at least this
# fraction of a loan before it may co-fund it.
MIN_PARTIAL_FRACTION = 0.20

# ---- Added in Week 5 (loan simulation) -----------------------------------
LATE_FEE_RATE = 0.02        # late fee = 2% of the instalment
LATE_DAYS = 7               # a late instalment is paid 7 days after the due date
# probability of (paid on time, paid late, defaulted) for each instalment
REPAYMENT_PROBABILITIES = {
    "low": (0.92, 0.07, 0.01),
    "medium": (0.80, 0.15, 0.05),
    "high": (0.65, 0.22, 0.13),
}
```

[NEW] scoring.py

```
"""Week 3 - transparent borrower-risk scoring and lender/borrower compatibility.

1. RiskScorer           : rule-based risk score 0-100 (higher = safer) + class low/medium/high
2. CompatibilityScorer  : hard-constraint checks + Risk/Amount/Term/Return fit (0-100)
3. score_all()          : scores every (active lender x pending request) pair and stores the
                          components and a plain-English explanation in the database.
"""
import json
import time
from dataclasses import dataclass, field

import audit
import config
import profile_service
from models import calculate_emi


# ---------------------------------------------------------------------------
# 1. Borrower risk
# ---------------------------------------------------------------------------
@dataclass
class RiskResult:
    score: float
    category: str
    components: dict
    explanation: str
    rate: float = 0.0


class RiskScorer:
    """Rule-based risk score.  Every component is shown to the user."""

    def classify(self, score):
        if score >= config.LOW_RISK_MIN_SCORE:
            return "low"
        if score >= config.MEDIUM_RISK_MIN_SCORE:
```

```
            return "medium"
        return "high"

    def score(self, borrower, request):
        w = config.RISK_SCORE_WEIGHTS
        credit = (min(max(borrower.credit_score, 300), 900) - 300) / 600 * w["credit"]
        dti = borrower.debt_to_income()
        dti_pts = (1 - min(dti / config.MAX_DEBT_TO_INCOME, 1)) * w["dti"]
        history_pts = min(borrower.credit_history_years, 10) / 10 * w["history"]
        lti = request.amount / (borrower.monthly_income * 12)
        lti_pts = (1 - min(lti, 1)) * w["loan_to_income"]
        components = {"credit": round(credit, 2), "dti": round(dti_pts, 2),
                      "history": round(history_pts, 2), "loan_to_income": round(lti_pts, 2)}
        total = round(sum(components.values()), 2)
        category = self.classify(total)
        rate = config.RATE_BY_RISK[category]
        explanation = (
            f"Risk score {total}/100 -> {category.upper()} risk (low >= {config.LOW_RISK_MIN_SCORE}, "
            f"medium >= {config.MEDIUM_RISK_MIN_SCORE}). "
            f"Credit score {borrower.credit_score}: {components['credit']}/{w['credit']}. "
            f"Debt-to-income {dti:.0%}: {components['dti']}/{w['dti']}. "
            f"Credit history {borrower.credit_history_years:g} years: {components['history']}/{w['history']}. "
            f"Loan is {lti:.0%} of annual income: {components['loan_to_income']}/{w['loan_to_income']}. "
            f"Offered interest rate for {category} risk: {rate:g}% per year.")
        return RiskResult(total, category, components, explanation, rate)


# ---------------------------------------------------------------------------
# 2. Pair compatibility
# ---------------------------------------------------------------------------
@dataclass
class PairResult:
    eligible: bool
    score: float = 0.0
    risk_fit: float = 0.0
    amount_fit: float = 0.0
    term_fit: float = 0.0
    return_fit: float = 0.0
    violations: list = field(default_factory=list)      # list of (code, message)
    explanation: str = ""

    @property
    def violation_codes(self):
        return [code for code, _ in self.violations]


def _centrality(value, low, high, floor):
    """1.0 at the middle of [low, high], falling to `floor` at the edges."""
    half = (high - low) / 2
    if half == 0:
        return 1.0
    return 1 - (1 - floor) * abs(value - (low + high) / 2) / half


class CompatibilityScorer:
    def __init__(self, allow_partial=False):
        self.allow_partial = allow_partial

    # ---- hard constraints ------------------------------------------------
    def check_constraints(self, lender, borrower, request, risk):
        v = []
        if not lender.is_matchable():
            v.append(("LENDER_INACTIVE", "Lender profile is not active."))
        if not borrower.is_matchable():
            v.append(("BORROWER_INACTIVE", "Borrower profile is not active."))
        if not lender.accepts_risk(risk.category):
            v.append(("RISK_APPETITE", f"Borrower is {risk.category} risk but the lender only accepts up to "
                                       f"{lender.risk_appetite} risk."))
        if not lender.accepts_amount(request.amount):
            v.append(("AMOUNT_RANGE", f"Loan amount {request.amount:,.0f} is outside the lender's range "
                                      f"{lender.min_loan_amount:,.0f}-{lender.max_loan_amount:,.0f}."))
        needed = request.amount * (config.MIN_PARTIAL_FRACTION if self.allow_partial else 1)
        if not lender.can_fund(needed):
            v.append(("INSUFFICIENT_FUNDS", f"Lender has {lender.available_amount:,.0f} available but "
                                            f"{needed:,.0f} is needed."))
        if not lender.accepts_duration(request.term_months):
            v.append(("DURATION_RANGE", f"Term {request.term_months} months is outside the lender's range "
                                        f"{lender.min_duration_months}-{lender.max_duration_months} months."))
        if not lender.accepts_rate(risk.rate):
            v.append(("RETURN_RANGE", f"Rate {risk.rate:g}% for {risk.category} risk is outside the lender's "
                                      f"expected return {lender.min_expected_return:g}-"
                                      f"{lender.max_expected_return:g}%."))
        emi = request.monthly_instalment(risk.rate)
        ratio = (borrower.existing_obligations + emi) / borrower.monthly_income
        if ratio > config.MAX_DEBT_TO_INCOME:
            v.append(("AFFORDABILITY", f"Obligations plus the new instalment ({emi:,.0f}) are {ratio:.0%} of "
                                       f"income, above the {config.MAX_DEBT_TO_INCOME:.0%} limit."))
        return v

    # ---- soft scores -----------------------------------------------------
    def fits(self, lender, request, risk):
        diff = lender.risk_level - config.RISK_ORDER[risk.category]
        risk_fit = {0: 1.0, 1: 0.7}.get(diff, 0.4)
        band = _centrality(request.amount, lender.min_loan_amount, lender.max_loan_amount, 0.5)
        capacity = 1 - 0.5 * min(1.0, request.amount / lender.available_amount)
        amount_fit = (band + capacity) / 2
        term_fit = _centrality(request.term_months, lender.min_duration_months, lender.max_duration_months, 0.6)
        span = lender.max_expected_return - lender.min_expected_return
        return_fit = 1.0 if span == 0 else 0.5 + 0.5 * (risk.rate - lender.min_expected_return) / span
        return risk_fit, amount_fit, term_fit, return_fit
```

```
    def evaluate(self, lender, borrower, request, risk):
        violations = self.check_constraints(lender, borrower, request, risk)
        if violations:
            text = "REJECTED: " + " ".join(message for _, message in violations)
            return PairResult(False, violations=violations, explanation=text)
        w = config.SCORE_WEIGHTS
        rf, af, tf, ef = self.fits(lender, request, risk)
        pts = {"risk_fit": round(rf * w["risk_fit"], 2), "amount_fit": round(af * w["amount_fit"], 2),
               "term_fit": round(tf * w["term_fit"], 2), "return_fit": round(ef * w["return_fit"], 2)}
        total = round(sum(pts.values()), 2)
        text = (f"ELIGIBLE: compatibility {total}/100 = risk fit {pts['risk_fit']}/{w['risk_fit']} "
                f"(lender appetite {lender.risk_appetite}, borrower {risk.category}) + amount fit "
                f"{pts['amount_fit']}/{w['amount_fit']} (loan {request.amount:,.0f} vs lender band and funds) + "
                f"term fit {pts['term_fit']}/{w['term_fit']} ({request.term_months} months) + return fit "
                f"{pts['return_fit']}/{w['return_fit']} (rate {risk.rate:g}% in lender's "
                f"{lender.min_expected_return:g}-{lender.max_expected_return:g}% range).")
        return PairResult(True, total, pts["risk_fit"], pts["amount_fit"], pts["term_fit"], pts["return_fit"],
                          [], text)


# ---------------------------------------------------------------------------
# 3. Score every pair and store the result
# ---------------------------------------------------------------------------
def reset_unaccepted_matches(conn):
    """Forget earlier unaccepted results so scoring/matching can be re-run."""
    conn.execute("UPDATE loan_requests SET status = 'pending' WHERE status = 'matched'")
    conn.execute("DELETE FROM matches WHERE status IN ('recommended', 'rejected', 'expired')")


def score_all(conn, allow_partial=False):
    """Score all active lenders against all pending requests.  Returns a summary dict."""
    started = time.perf_counter()
    reset_unaccepted_matches(conn)
    lenders = profile_service.list_lenders(conn, profile_status="active")
    requests = [r for r in conn.execute(
        "SELECT request_id FROM loan_requests WHERE status = 'pending' ORDER BY request_id")]
    scorer, risk_scorer = CompatibilityScorer(allow_partial), RiskScorer()
    match_rows, eligible, rejected, skipped = [], 0, 0, 0

    for row in requests:
        request = profile_service.get_request(conn, row["request_id"])
        borrower = profile_service.get_borrower(conn, request.borrower_id)
        if borrower is None or not borrower.is_matchable():
            skipped += 1
            continue
        risk = risk_scorer.score(borrower, request)
        conn.execute("DELETE FROM risk_scores WHERE request_id = ?", (request.request_id,))
        conn.execute("INSERT INTO risk_scores(borrower_id, request_id, score, risk_category, components, explanation)"
                     " VALUES (?, ?, ?, ?, ?, ?)",
                     (request.borrower_id, request.request_id, risk.score, risk.category,
                      json.dumps(risk.components), risk.explanation))
        for lender in lenders:
            result = scorer.evaluate(lender, borrower, request, risk)
            if result.eligible:
                eligible += 1
            else:
                rejected += 1
            match_rows.append((lender.lender_id, request.request_id, result.score, result.risk_fit,
                               result.amount_fit, result.term_fit, result.return_fit, 0.0,
                               "recommended" if result.eligible else "rejected", result.explanation))
    conn.executemany(
        "INSERT INTO matches(lender_id, request_id, compatibility_score, risk_fit, amount_fit, term_fit,"
        " return_fit, allocated_amount, status, explanation) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", match_rows)
    audit.log(conn, "system", None, "SCORING_RUN", None,
              details=f"{len(match_rows)} pairs, {eligible} eligible, {rejected} rejected")
    conn.commit()
    return {"requests_scored": len(requests) - skipped, "lenders": len(lenders), "pairs": len(match_rows),
            "eligible": eligible, "rejected": rejected, "seconds": round(time.perf_counter() - started, 4)}
```

[NEW] queries.py

```
"""Read-only queries used by several screens (Week 3)."""

_PAIR_SQL = """
SELECT m.match_id, m.lender_id, ul.full_name AS lender, m.request_id, ub.full_name AS borrower,
       r.amount, r.term_months, m.compatibility_score AS score, m.risk_fit, m.amount_fit, m.term_fit,
       m.return_fit, m.allocated_amount, m.status, m.explanation
FROM matches m
JOIN lenders l ON l.lender_id = m.lender_id JOIN users ul ON ul.user_id = l.user_id
JOIN loan_requests r ON r.request_id = m.request_id
JOIN borrowers b ON b.borrower_id = r.borrower_id JOIN users ub ON ub.user_id = b.user_id
"""


def list_pairs(conn, status=None, request_id=None, lender_id=None):
    where, params = [], []
    if status:
        where.append("m.status = ?")
        params.append(status)
    if request_id:
        where.append("m.request_id = ?")
        params.append(request_id)
    if lender_id:
        where.append("m.lender_id = ?")
        params.append(lender_id)
    sql = _PAIR_SQL + (" WHERE " + " AND ".join(where) if where else "")
    sql += " ORDER BY m.request_id, m.compatibility_score DESC, m.match_id"
    return [dict(r) for r in conn.execute(sql, params)]


def list_risk_scores(conn):
    sql = ("SELECT s.request_id, u.full_name AS borrower, r.amount, r.term_months, s.score,"
           " s.risk_category, s.explanation FROM risk_scores s"
           " JOIN loan_requests r ON r.request_id = s.request_id"
           " JOIN borrowers b ON b.borrower_id = s.borrower_id JOIN users u ON u.user_id = b.user_id"
           " ORDER BY s.request_id")
    return [dict(r) for r in conn.execute(sql)]
```

[NEW] routes/scoring.py

```
"""Week 3 routes - run scoring, risk explanation screen, scored-pair screen."""
from flask import Blueprint, flash, redirect, render_template, request, url_for

import queries
import scoring
from utils import db_conn

bp = Blueprint("scoring", __name__, url_prefix="/scoring")


@bp.route("/run", methods=["POST"])
def run():
    partial = request.args.get("partial") == "1"
    with db_conn() as conn:
        s = scoring.score_all(conn, allow_partial=partial)
    flash(f"Scored {s['pairs']} pairs: {s['eligible']} eligible, {s['rejected']} rejected "
          f"({s['seconds']} s).", "success")
    return redirect(url_for("scoring.pairs"))


@bp.route("/risk")
def risk_scores():
    with db_conn() as conn:
        rows = queries.list_risk_scores(conn)
    columns = [("request_id", "Request"), ("borrower", "Borrower"), ("amount", "Amount"),
               ("term_months", "Term"), ("score", "Score /100"), ("risk_category", "Risk class"),
               ("explanation", "Why")]
    return render_template("table.html", title="Borrower risk scores", columns=columns, rows=rows,
                           intro="Higher score = safer borrower. Run scoring first if this table is empty.",
                           top_actions=[{"label": "Run scoring", "method": "post",
                                         "url": url_for("scoring.run")}])


@bp.route("/pairs")
def pairs():
    status = request.args.get("status")
    request_id = request.args.get("request_id", type=int)
    with db_conn() as conn:
        rows = queries.list_pairs(conn, status=status, request_id=request_id)
    columns = [("request_id", "Req"), ("borrower", "Borrower"), ("lender", "Lender"), ("amount", "Amount"),
               ("score", "Score"), ("risk_fit", "Risk"), ("amount_fit", "Amt"), ("term_fit", "Term"),
               ("return_fit", "Ret"), ("status", "Status"), ("explanation", "Why recommended / rejected")]
    return render_template(
        "table.html", title="Scored lender-borrower pairs", columns=columns, rows=rows,
        intro="Filter with ?status=recommended, ?status=rejected or ?status=expired (also ?request_id=N).",
        top_actions=[{"label": "Run scoring", "method": "post", "url": url_for("scoring.run")},
                     {"label": "Run scoring (partial funding)", "method": "post", "style": "secondary",
                      "url": url_for("scoring.run", partial=1)}])
```

[NEW] *demo_week3.py*

```
"""Week 3 demo: risk scores, hard-constraint rejections and match explanations."""
import demo_data
import queries
import scoring

conn = demo_data.fresh_demo_db()
summary = scoring.score_all(conn)
print("Scoring summary:", summary)

print("\n=== Borrower risk scores ===")
for r in queries.list_risk_scores(conn):
    print(f"Request #{r['request_id']} {r['borrower']:<12} amount={r['amount']:>9,.0f}  "
          f"score={r['score']:>6}  -> {r['risk_category'].upper()}")

print("\n=== Eligible lender candidates per request (best first) ===")
for p in queries.list_pairs(conn, status="recommended"):
    print(f"Request #{p['request_id']} {p['borrower']:<12} <- {p['lender']:<19} score={p['score']:>6} "
          f"(risk {p['risk_fit']} + amount {p['amount_fit']} + term {p['term_fit']} + return {p['return_fit']})")

print("\n=== Why were pairs rejected? (request #4 - Anita Das) ===")
for p in queries.list_pairs(conn, status="rejected", request_id=4)[:2]:
    print(f"{p['lender']}: {p['explanation']}")

print("\n=== Example explanation of a recommendation ===")
best = queries.list_pairs(conn, status="recommended", request_id=3)[0]
print(best["explanation"])
print("\n=== Example risk explanation ===")
print(queries.list_risk_scores(conn)[0]["explanation"])
conn.close()
```

[NEW] *tests/test_week3.py*

```
"""Week 3 tests: risk scoring, hard constraints, compatibility scores, stored explanations."""
import json
import os
import sys
import unittest

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import config
import demo_data
import queries
import scoring
from models import Borrower, Lender, LoanRequest


def lender(**kw):
    base = dict(full_name="Len Der", email="l@x.com", risk_appetite="medium", available_amount=100000,
                min_loan_amount=5000, max_loan_amount=80000, min_duration_months=6, max_duration_months=24,
                min_expected_return=9, max_expected_return=15, profile_status="active")
    base.update(kw)
    return Lender(**base)


def borrower(**kw):
    base = dict(full_name="Bor Rower", email="b@x.com", monthly_income=60000, existing_obligations=6000,
                credit_score=700, credit_history_years=5, profile_status="active")
    base.update(kw)
    return Borrower(**base)


def request(amount=30000, term=12):
    return LoanRequest(1, amount, term)


class RiskTests(unittest.TestCase):
    def test_classes_and_weights(self):
        self.assertEqual(sum(config.RISK_SCORE_WEIGHTS.values()), 100)
        s = scoring.RiskScorer()
        self.assertEqual(s.classify(70), "low")
        self.assertEqual(s.classify(69.99), "medium")
        self.assertEqual(s.classify(45), "medium")
        self.assertEqual(s.classify(44.99), "high")

    def test_better_borrower_scores_higher(self):
        s = scoring.RiskScorer()
        good = s.score(borrower(credit_score=820, existing_obligations=1000, credit_history_years=10), request())
        bad = s.score(borrower(credit_score=450, existing_obligations=30000, credit_history_years=0.5), request())
        self.assertGreater(good.score, bad.score)
        self.assertEqual((good.category, bad.category), ("low", "high"))
        self.assertTrue(0 <= bad.score <= good.score <= 100)

    def test_components_add_up(self):
        r = scoring.RiskScorer().score(borrower(), request())
        self.assertAlmostEqual(sum(r.components.values()), r.score, places=1)
        self.assertIn("Offered interest rate", r.explanation)


class ConstraintTests(unittest.TestCase):
    def codes(self, l=None, b=None, r=None, partial=False, category="medium"):
        l, b, r = l or lender(), b or borrower(), r or request()
        risk = scoring.RiskResult(60, category, {}, "", config.RATE_BY_RISK[category])
        return scoring.CompatibilityScorer(partial).evaluate(l, b, r, risk).violation_codes

    def test_ok_pair_has_no_violations(self):
        self.assertEqual(self.codes(), [])
    def test_risk_appetite_mismatch(self):
```

```
        self.assertIn("RISK_APPETITE", self.codes(l=lender(risk_appetite="low")))

    def test_amount_outside_lender_range(self):
        self.assertIn("AMOUNT_RANGE", self.codes(r=request(amount=90000)))

    def test_loan_exceeds_available_funds_and_partial_mode(self):
        small = lender(available_amount=20000)
        self.assertIn("INSUFFICIENT_FUNDS", self.codes(l=small))
        self.assertNotIn("INSUFFICIENT_FUNDS", self.codes(l=small, partial=True))   # 20000 >= 20% of 30000

    def test_term_mismatch(self):
        self.assertIn("DURATION_RANGE", self.codes(r=request(term=48)))

    def test_return_range_mismatch(self):
        self.assertIn("RETURN_RANGE", self.codes(l=lender(min_expected_return=15, max_expected_return=18)))

    def test_affordability(self):
        self.assertIn("AFFORDABILITY", self.codes(b=borrower(existing_obligations=35000)))

    def test_inactive_profiles(self):
        self.assertIn("LENDER_INACTIVE", self.codes(l=lender(profile_status="pending")))
        self.assertIn("BORROWER_INACTIVE", self.codes(b=borrower(profile_status="inactive")))

    def test_multiple_violations_are_all_reported(self):
        codes = self.codes(l=lender(risk_appetite="low"), r=request(amount=90000, term=48))
        self.assertTrue({"RISK_APPETITE", "AMOUNT_RANGE", "DURATION_RANGE"} <= set(codes))


class CompatibilityTests(unittest.TestCase):
    def evaluate(self, l, category="medium"):
        risk = scoring.RiskResult(60, category, {}, "", config.RATE_BY_RISK[category])
        return scoring.CompatibilityScorer().evaluate(l, borrower(), request(), risk)

    def test_score_is_sum_of_components(self):
        r = self.evaluate(lender())
        self.assertTrue(r.eligible)
        self.assertAlmostEqual(r.score, r.risk_fit + r.amount_fit + r.term_fit + r.return_fit, places=1)
        self.assertTrue(0 < r.score <= 100)

    def test_exact_risk_match_beats_looser_appetite(self):
        exact = self.evaluate(lender(risk_appetite="medium"))
        loose = self.evaluate(lender(risk_appetite="high"))
        self.assertGreater(exact.risk_fit, loose.risk_fit)
        self.assertEqual(exact.risk_fit, config.SCORE_WEIGHTS["risk_fit"])

    def test_explanations_present(self):
        self.assertTrue(self.evaluate(lender()).explanation.startswith("ELIGIBLE"))
        self.assertTrue(self.evaluate(lender(risk_appetite="low")).explanation.startswith("REJECTED"))


class StoredScoringTests(unittest.TestCase):
    def setUp(self):
        self.conn = demo_data.fresh_demo_db(os.path.join(config.BASE_DIR, "data", "test_w3.db"))

    def tearDown(self):
        self.conn.close()

    def test_every_pair_is_stored_with_explanation(self):
        s = scoring.score_all(self.conn)
        self.assertEqual(s["pairs"], 35)
        self.assertEqual(s["eligible"] + s["rejected"], 35)
        rows = queries.list_pairs(self.conn)
        self.assertEqual(len(rows), 35)
        self.assertTrue(all(r["explanation"] for r in rows))
        self.assertTrue(all(r["score"] == 0 for r in rows if r["status"] == "rejected"))

    def test_risk_scores_stored_with_json_components(self):
        scoring.score_all(self.conn)
        row = self.conn.execute("SELECT * FROM risk_scores WHERE request_id = 1").fetchone()
        self.assertEqual(row["risk_category"], "low")
        self.assertEqual(set(json.loads(row["components"])), {"credit", "dti", "history", "loan_to_income"})

    def test_no_compatible_lender_for_unaffordable_request(self):
        scoring.score_all(self.conn)
        self.assertEqual(queries.list_pairs(self.conn, status="recommended", request_id=4), [])

    def test_rerun_does_not_duplicate(self):
        scoring.score_all(self.conn)
        scoring.score_all(self.conn)
        self.assertEqual(self.conn.execute("SELECT COUNT(*) FROM matches").fetchone()[0], 35)
        self.assertEqual(self.conn.execute("SELECT COUNT(*) FROM risk_scores").fetchone()[0], 7)

    def test_partial_mode_creates_extra_candidates(self):
        full = scoring.score_all(self.conn)["eligible"]
        partial = scoring.score_all(self.conn, allow_partial=True)["eligible"]
        self.assertGreater(partial, full)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Screenshots to attach to your submission

- /scoring/risk page showing score, class and explanation of each request.
- /scoring/pairs page showing an ELIGIBLE and a REJECTED explanation (use ?status=rejected).

• Terminal output of demo_week3.py and the 20 passing tests.